

Lộ trình luyện tập code Memory cho Agent

Từ cơ bản tới nâng cao - lý thuyết, cơ chế, trực quan hóa và thực hành với Qwen local LLM

Tài liệu bao gồm toàn bộ nội dung hướng dẫn, cấu trúc project, giải thích input/output từng file và toàn bộ code mẫu dạng .py.

Định dạng: PDF kỹ thuật một cột, code block dùng syntax theme để nổi bật và tự xuống dòng để tránh tràn trang.

Short-term

SQLite

Embedding

Hybrid Retrieval

Evaluation

Mục lục nhanh

- 1. Tư duy nền tảng: Memory trong agent là gì?
- 2. Bản đồ cơ chế memory agent
- 3. Lộ trình học từ cơ bản tới nâng cao
- 4. Cấu trúc project .py
- 5. Cài đặt môi trường
- 6. Code từng file
- 7. Test thực hành
- 8. Bài tập nâng cấp theo từng level
- 9. Những lỗi tư duy thường gặp khi học memory
- 10. Lộ trình học để xuất trong 10 buổi
- 11. Thứ tự nên code
- 12. Kết luận định hướng

Ghi chú định dạng

Bản này giữ nguyên nội dung tài liệu gốc, chỉ cải thiện màu sắc, tăng cỡ tiêu đề và đổi code block sang syntax theme để nổi bật hơn, đồng thời giữ ngắt trang an toàn hơn.

Lộ trình luyện tập code Memory cho Agent

Tài liệu này trình bày một lộ trình học và thực hành memory cho AI agent theo hướng đi từ nền tảng đến nâng cao. Mục tiêu không chỉ là biết dùng thư viện, mà là hiểu sâu cơ chế: memory được lưu thế nào, truy xuất thế nào, đưa vào prompt ra sao, và làm sao đánh giá memory có thật sự hữu ích.

Mục tiêu cuối là tự xây được một agent có:

- Short-term memory: nhớ vài lượt hội thoại gần nhất.
- Long-term memory: lưu xuống SQLite.
- Semantic memory: tìm lại ký ức bằng embedding.
- Hybrid memory: kết hợp vector search + keyword search + recency + importance.
- Memory writer: tự quyết định thông tin nào nên nhớ.
- Memory evaluator: kiểm tra memory có giúp trả lời tốt hơn không.

Model local thật được đề xuất là Qwen/Qwen3-4B-Instruct-2507 . Khi dùng Qwen3, nên cài `transformers>=4.51.0` để tránh lỗi tương thích như `KeyError: 'qwen3'` . Qwen dùng chuẩn chat template của Transformers để biến `messages=[{"role":..., "content":...}]` thành prompt đúng định dạng cho model chat.

1 Tư duy nền tảng: Memory trong agent là gì?

Trong agent, context và memory không giống nhau.

Context là phần text được nhét vào prompt hiện tại. Nó sống trong một lần gọi LLM.

Memory là hệ thống lưu, truy xuất, lọc, tóm tắt và đưa thông tin cũ trở lại prompt khi cần.

Ví dụ:

```
text
1 User: Từ nay tôi muốn bạn giải thích code theo kiểu dễ hiểu cho người mới.
```

Nếu chỉ để trong context, vài lượt sau có thể bị trôi mất.

Nếu lưu vào memory, agent có thể nhớ lâu dài rằng:

```
text
1 User preference: thích giải thích code theo kiểu dễ hiểu cho người mới.
```

2

Bản đồ cơ chế memory agent

```
mermaid
graph TD
    A[User input] --> B[Short-term memory]
    A --> C[Memory retriever]
    C --> D[Keyword search SQLite FTS5]
    C --> E[Vector search embedding]
    C --> F[Recent memory]
    D --> G[Memory ranker]
    E --> G
    F --> G
    G --> H[Prompt builder]
    B --> H
    H --> I[Qwen local LLM]
    I --> J[Assistant answer]
    A --> K[Memory writer]
    J --> K
    K --> L[SQLite memory store]
    K --> M[Embedding store]
```

Luồng hoạt động:

1. User nhập câu hỏi.
2. Agent lấy vài lượt hội thoại gần nhất.
3. Agent tìm memory liên quan trong SQLite/vector store.
4. Agent build prompt gồm system prompt + memory liên quan + history gần nhất + câu hỏi mới.
5. Qwen sinh câu trả lời.
6. Agent lưu lại lượt hội thoại.
7. Nếu câu user có thông tin dài hạn, agent lưu thành memory quan trọng.

3

Lộ trình học từ cơ bản tới nâng cao

Giai đoạn 1 - Short-term memory

Mục tiêu: hiểu agent nhớ hội thoại gần nhất như thế nào.

Cơ chế:

```
python
1 history = [
2     {"role": "user", "content": "..."},
3     {"role": "assistant", "content": "..."}
4 ]
```

Mỗi lần user hỏi, ta gửi lại một phần history vào model.

Bài thực hành:

- Giới hạn max_history_turns = 4 .
- Test xem agent có nhớ tên bạn trong 2-3 lượt không.
- Tăng số lượt history và quan sát tốc độ chậm đi.

Giai đoạn 2 - Persistent memory với SQLite

Mục tiêu: nhớ được qua nhiều lần chạy chương trình.

Cơ chế:

```
text
1 MemoryItem
2 - id
3 - content
4 - memory_type
5 - importance
6 - tags
7 - created_at
8 - source
```

Bài thực hành:

- Lưu mọi lượt hội thoại xuống SQLite.
- Tắt chương trình, mở lại, agent vẫn xem được lịch sử cũ.
- Thêm lệnh /memories để xem memory đã lưu.

Giai đoạn 3 - Semantic memory bằng embedding

Mục tiêu: user hỏi khác chữ nhưng agent vẫn tìm đúng memory.

Ví dụ memory cũ:

```
text
1 User đang học ngành ETE tại DUT.
```

User hỏi:

```
1 Tôi nên học môn nào để hợp với ngành của tôi?
```

Keyword search có thể không bắt được vì không có chữ “ETE” hoặc “DUT”. Embedding giúp tìm theo nghĩa.

Sentence Transformers có thể dùng để biến câu thành vector cố định và dùng cho semantic search, clustering, similarity search. Model sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 tạo vector 384 chiều và hỗ trợ semantic search đa ngôn ngữ, phù hợp cho tiếng Việt ở mức bắt đầu.

Giai đoạn 4 - Hybrid memory

Mục tiêu: tìm memory tốt hơn bằng cách kết hợp:

```
1 final_score =
2     semantic_score * 0.55
3 + keyword_bonus * 0.20
4 + importance * 0.15
5 + recency_score * 0.10
```

Vì sao cần hybrid?

- Vector search giỏi tìm theo nghĩa.
- Keyword search giỏi tìm đúng tên riêng, mã lỗi, tên file.
- Recency giúp ưu tiên thông tin mới.
- Importance giúp ưu tiên thông tin user bảo “nhớ từ nay”.

SQLite FTS5 có hỗ trợ full-text search và hàm `bm25()` để xếp hạng kết quả; trong SQLite FTS5, điểm BM25 càng nhỏ thì match càng tốt khi `ORDER BY bm25(...)`.

Giai đoạn 5 - Memory writer

Mục tiêu: không lưu rác.

Không phải câu nào cũng nên lưu. Ví dụ:

Không nên lưu:

```
1 Hôm nay mấy giờ rồi?
```

Nên lưu:

```

text
1 Từ nay khi giải code, hãy giải thích input/output từng file.
    
```

Memory writer có thể có 2 lớp:

```

text
1 Rule-based writer:
2 - Nếu có "từ nay", "nhớ rằng", "sau này", "going forward" → lưu.
3
4 LLM-based writer:
5 - Gửi user input + assistant output cho Qwen.
6 - Qwen trả JSON: có nên lưu không, lưu loại gì, độ quan trọng bao nhiêu.
    
```

Giai đoạn 6 - Reflection memory

Mục tiêu: agent không chỉ nhớ từng câu, mà còn rút ra kết luận.

Ví dụ sau nhiều lượt:

```

text
1 User thường học theo kiểu làm project thực tế.
2 User thích giải thích chi tiết cơ chế từng file.
3 User đang tập trung vào Agentic AI và local LLM.
    
```

Đây là memory cấp cao hơn, gọi là reflection hoặc summary memory.

Giai đoạn 7 - Evaluation

Mục tiêu: biết memory có thật sự giúp ích không.

Bạn tạo bộ test:

```

text
1 Turn 1: Tôi học ngành ETE ở DUT.
2 Turn 2: Tôi muốn chuyển hướng sang vi mạch.
3 Turn 3: Gợi ý lộ trình học phù hợp với định hướng của tôi.
    
```

So sánh:

```

text
1 Without memory: câu trả lời chung chung.
2 With memory: câu trả lời bám ETE + vi mạch + DUT.
    
```

4 Cấu trúc project `.py`

```

1 agent_memory_lab/
2 |
3 |— requirements.txt
4 |— config.py
5 |— main.py
6 |
7 |— llm/
8 |   |— qwen_local.py
9 |
10 |— memory/
11 |   |— schema.py
12 |   |— embedder.py
13 |   |— sqlite_memory.py
14 |   |— memory_manager.py
15 |
16 |— agent/
17 |   |— prompt_builder.py
18 |   |— memory_agent.py
19 |
20 |— scripts/
21 |   |— visualize_memory.py

```

5 Cài đặt môi trường

requirements.txt

```

1 torch
2 transformers>=4.51.0
3 accelerate
4 sentence-transformers
5 numpy
6 rich

```

Cài:

```

1 pip install -r requirements.txt

```

Với RTX 3060 12GB, Qwen 4B thường khả thi hơn khi chạy trên WSL/Linux. Nếu bị OOM, giảm max_new_tokens , đóng app GPU khác, hoặc chuyển sang bản nhỏ hơn trước khi tối ưu quantization.

6

Code từng file

6.1. config.py

Input: không có.

Output: cấu hình dùng chung toàn project.

```
python
1 from dataclasses import dataclass
2
3
4 @dataclass
5 class Settings:
6     model_name: str = "Qwen/Qwen3-4B-Instruct-2507"
7     embedding_model_name: str = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
8
9     db_path: str = "memory.db"
10
11     max_history_turns: int = 4
12     max_retrieved_memories: int = 5
13
14     max_new_tokens: int = 512
15     temperature: float = 0.3
16
17
18 settings = Settings()
```

Cơ chế:

- model_name : model LLM thật.
- embedding_model_name : model biến câu thành vector.
- db_path : nơi lưu long-term memory.
- max_history_turns : số lượt hội thoại gần nhất đưa vào prompt.
- max_retrieved_memories : số memory cũ được inject vào prompt.

6.2. llm/qwen_local.py

Input:

```
python
1 messages = [
2     {"role": "system", "content": "..."},
3     {"role": "user", "content": "..."}
4 ]
```

Output:

python

```
1 assistant_text: str
```

python

```
1 import torch
2 from transformers import AutoModelForCausalLM, AutoTokenizer
3
4 from config import settings
5
6
7 class QwenLocalLLM:
8     def __init__(self):
9         self.tokenizer = AutoTokenizer.from_pretrained(settings.model_name)
10
11         self.model = AutoModelForCausalLM.from_pretrained(
12             settings.model_name,
13             torch_dtype="auto",
14             device_map="auto",
15         )
16
17         self.model.eval()
18
19     def generate(self, messages: list[dict]) -> str:
20         text = self.tokenizer.apply_chat_template(
21             messages,
22             tokenize=False,
23             add_generation_prompt=True,
24         )
25
26         device = next(self.model.parameters()).device
27
28         inputs = self.tokenizer(
29             [text],
30             return_tensors="pt",
31         ).to(device)
32
33         with torch.no_grad():
34             output_ids = self.model.generate(
35                 **inputs,
36                 max_new_tokens=settings.max_new_tokens,
37                 temperature=settings.temperature,
38                 do_sample=settings.temperature > 0,
39                 pad_token_id=self.tokenizer.eos_token_id,
40             )
41
42         generated_ids = output_ids[0][inputs.input_ids.shape[-1]:]
43
44         answer = self.tokenizer.decode(
45             generated_ids,
46             skip_special_tokens=True,
47         )
48
49         return answer.strip()
```

Cơ chế:

1. `AutoTokenizer.from_pretrained()` tải tokenizer của Qwen.
2. `AutoModelForCausalLM.from_pretrained()` tải model sinh văn bản.

3. `apply_chat_template()` biến list messages thành prompt đúng format.
4. `generate()` sinh token mới.
5. Cắt bỏ phần input ban đầu, chỉ decode phần model vừa sinh.

6.3. `memory/schema.py`

Input: dữ liệu text cần lưu.

Output: object chuẩn hóa `MemoryItem` .

```
python
1 from dataclasses import dataclass, field
2 from datetime import datetime
3 import uuid
4
5
6 @dataclass
7 class MemoryItem:
8     content: str
9     memory_type: str = "turn"
10    importance: float = 0.5
11    tags: list[str] = field(default_factory=list)
12    source: str = "conversation"
13    metadata: dict = field(default_factory=dict)
14    id: str = field(default_factory=lambda: str(uuid.uuid4()))
15    created_at: str = field(default_factory=lambda: datetime.utcnow().isoformat())
16
17    def to_display_text(self) -> str:
18        tags = ", ".join(self.tags) if self.tags else "no-tags"
19        return (
20            f"[{self.memory_type}] "
21            f"importance={self.importance:.2f} "
22            f"tags={tags}\n"
23            f"{self.content}"
24        )
```

Cơ chế:

- Mỗi memory là một đơn vị độc lập.
- `memory_type` giúp phân loại: `turn` , `preference` , `fact` , `reflection` .
- `importance` dùng khi ranking.
- `tags` giúp lọc hoặc debug.

6.4. `memory/embedder.py`

Input:

```
python
1 text: str
```

Output:

python

```
1 np.ndarray vector
```

python

```
1 import numpy as np
2 from sentence_transformers import SentenceTransformer
3
4 from config import settings
5
6
7 class Embedder:
8     def __init__(self):
9         self.model = SentenceTransformer(settings.embedding_model_name)
10
11     def encode_one(self, text: str) -> np.ndarray:
12         vector = self.model.encode(
13             [text],
14             normalize_embeddings=True,
15         )[0]
16
17         return vector.astype("float32")
18
19     def encode_many(self, texts: list[str]) -> np.ndarray:
20         vectors = self.model.encode(
21             texts,
22             normalize_embeddings=True,
23         )
24
25         return vectors.astype("float32")
```

Cơ chế:

- Text được biến thành vector.
- `normalize_embeddings=True` giúp cosine similarity gần tương đương dot product.
- Hai câu gần nghĩa nhau sẽ có vector gần nhau hơn.

6.5. memory/sqlite_memory.py

Input:

python

```
1 MemoryItem + embedding vector
```

Output:

python

```
1 lưu xuống SQLite / trả kết quả search
```

```
python

1 import json
2 import re
3 import sqlite3
4 from pathlib import Path
5
6 import numpy as np
7
8 from memory.schema import MemoryItem
9
10
11 class SQLiteMemoryStore:
12     def __init__(self, db_path: str):
13         self.db_path = db_path
14         Path(db_path).parent.mkdir(parents=True, exist_ok=True) if Path(db_path).parent != Path(".") else None
15         self._init_db()
16
17     def _connect(self):
18         return sqlite3.connect(self.db_path)
19
20     def _init_db(self):
21         with self._connect() as conn:
22             cur = conn.cursor()
23
24             cur.execute("""
25             CREATE TABLE IF NOT EXISTS memories (
26                 id TEXT PRIMARY KEY,
27                 content TEXT NOT NULL,
28                 memory_type TEXT,
29                 importance REAL,
30                 tags_json TEXT,
31                 source TEXT,
32                 created_at TEXT,
33                 metadata_json TEXT,
34                 embedding BLOB,
35                 embedding_dim INTEGER
36             )
37             """)
38
39             cur.execute("""
40             CREATE VIRTUAL TABLE IF NOT EXISTS memory_fts
41             USING fts5(
42                 id UNINDEXED,
43                 content,
44                 tags
45             )
46             """)
47
48             conn.commit()
49
50     def add(self, item: MemoryItem, embedding: np.ndarray | None = None):
51         embedding_blob = None
52         embedding_dim = None
53
54         if embedding is not None:
55             embedding = embedding.astype("float32")
56             embedding_blob = embedding.tobytes()
57             embedding_dim = int(embedding.shape[0])
58
59         with self._connect() as conn:
60             cur = conn.cursor()
61
62             cur.execute("""
63             INSERT OR REPLACE INTO memories (
```

python (tiếp)

```

64         id, content, memory_type, importance,
65         tags_json, source, created_at, metadata_json,
66         embedding, embedding_dim
67     )
68     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
69     """ , (
70         item.id,
71         item.content,
72         item.memory_type,
73         item.importance,
74         json.dumps(item.tags, ensure_ascii=False),
75         item.source,
76         item.created_at,
77         json.dumps(item.metadata, ensure_ascii=False),
78         embedding_blob,
79         embedding_dim,
80     ))
81
82     cur.execute(
83         "DELETE FROM memory_fts WHERE id = ?",
84         (item.id,),
85     )
86
87     cur.execute("""
88     INSERT INTO memory_fts (id, content, tags)
89     VALUES (?, ?, ?)
90     """ , (
91         item.id,
92         item.content,
93         " ".join(item.tags),
94     ))
95
96     conn.commit()
97
98     def get_by_id(self, memory_id: str) -> MemoryItem | None:
99         with self._connect() as conn:
100             cur = conn.cursor()
101             cur.execute("""
102             SELECT id, content, memory_type, importance,
103                     tags_json, source, created_at, metadata_json
104             FROM memories
105             WHERE id = ?
106             """ , (memory_id,))
107
108             row = cur.fetchone()
109
110             if not row:
111                 return None
112
113             return self._row_to_item(row)
114
115     def recent(self, limit: int = 5) -> list[MemoryItem]:
116         with self._connect() as conn:
117             cur = conn.cursor()
118             cur.execute("""
119             SELECT id, content, memory_type, importance,
120                     tags_json, source, created_at, metadata_json
121             FROM memories
122             ORDER BY created_at DESC
123             LIMIT ?
124             """ , (limit,))
125
126             rows = cur.fetchall()
127

```

python (tiếp)

```

128         return [self._row_to_item(row) for row in rows]
129
130     def keyword_search(self, query: str, limit: int = 5) -> list[tuple[MemoryItem, float]]:
131         safe_query = self._to_fts_query(query)
132
133         if not safe_query:
134             return []
135
136         with self._connect() as conn:
137             cur = conn.cursor()
138
139             cur.execute("""
140             SELECT id, bm25(memory_fts) AS rank
141             FROM memory_fts
142             WHERE memory_fts MATCH ?
143             ORDER BY rank
144             LIMIT ?
145             """, (safe_query, limit))
146
147             rows = cur.fetchall()
148
149             results = []
150
151             for memory_id, rank in rows:
152                 item = self.get_by_id(memory_id)
153                 if item:
154                     results.append((item, float(rank)))
155
156             return results
157
158     def vector_search(self, query_vector: np.ndarray, limit: int = 5) -> list[tuple[MemoryItem, float]]:
159         query_vector = query_vector.astype("float32")
160
161         with self._connect() as conn:
162             cur = conn.cursor()
163
164             cur.execute("""
165             SELECT id, content, memory_type, importance,
166                    tags_json, source, created_at, metadata_json,
167                    embedding, embedding_dim
168             FROM memories
169             WHERE embedding IS NOT NULL
170             """)
171
172             rows = cur.fetchall()
173
174             scored = []
175
176             for row in rows:
177                 embedding_blob = row[8]
178                 embedding_dim = row[9]
179
180                 memory_vector = np.frombuffer(
181                     embedding_blob,
182                     dtype="float32",
183                 ).reshape(embedding_dim)
184
185                 score = float(np.dot(query_vector, memory_vector))
186
187                 item = self._row_to_item(row[:8])
188                 scored.append((item, score))
189
190             scored.sort(key=lambda x: x[1], reverse=True)

```

```
python (tiếp)

191
192     return scored[:limit]
193
194     def all_memories(self) -> list[MemoryItem]:
195         with self._connect() as conn:
196             cur = conn.cursor()
197             cur.execute("""
198                 SELECT id, content, memory_type, importance,
199                     tags_json, source, created_at, metadata_json
200                 FROM memories
201                 ORDER BY created_at DESC
202             """)
203
204             rows = cur.fetchall()
205
206             return [self._row_to_item(row) for row in rows]
207
208     def _row_to_item(self, row) -> MemoryItem:
209         return MemoryItem(
210             id=row[0],
211             content=row[1],
212             memory_type=row[2],
213             importance=float(row[3]),
214             tags=json.loads(row[4]),
215             source=row[5],
216             created_at=row[6],
217             metadata=json.loads(row[7]),
218         )
219
220     def _to_fts_query(self, text: str) -> str:
221         tokens = re.findall(r"\w+", text.lower(), flags=re.UNICODE)
222         return " OR ".join(tokens[:10])
```

Cơ chế chính:

- Bảng memories lưu dữ liệu đầy đủ.
- Bảng memory_fts hỗ trợ keyword search.
- Embedding được lưu dạng BLOB .
- Khi vector search, ta load vector từ SQLite, tính dot product với query vector.

6.6. memory/memory_manager.py

Input:

```
python

1 user_text
2 assistant_text
```

Output:

```
python

1 retrieved memories / saved memories
```



```
python
1 from datetime import datetime, timezone
2
3 from config import settings
4 from memory.embedder import Embedder
5 from memory.schema import MemoryItem
6 from memory.sqlite_memory import SQLiteMemoryStore
7
8
9 class MemoryManager:
10     def __init__(self):
11         self.store = SQLiteMemoryStore(settings.db_path)
12         self.embedder = Embedder()
13
14     def save_turn(self, user_text: str, assistant_text: str):
15         content = f"User: {user_text}\nAssistant: {assistant_text}"
16
17         item = MemoryItem(
18             content=content,
19             memory_type="turn",
20             importance=0.4,
21             tags=["conversation"],
22         )
23
24         vector = self.embedder.encode_one(content)
25         self.store.add(item, vector)
26
27         explicit_memory = self._extract_explicit_memory(user_text)
28
29         if explicit_memory:
30             vector = self.embedder.encode_one(explicit_memory.content)
31             self.store.add(explicit_memory, vector)
32
33     def remember(self, content: str, memory_type: str = "fact", importance: float = 0.7, tags: list[str] | None = None):
34         item = MemoryItem(
35             content=content,
36             memory_type=memory_type,
37             importance=importance,
38             tags=tags or [],
39         )
40
41         vector = self.embedder.encode_one(content)
42         self.store.add(item, vector)
43
44     def retrieve(self, query: str, limit: int | None = None) -> list[MemoryItem]:
45         limit = limit or settings.max_retrieved_memories
46
47         query_vector = self.embedder.encode_one(query)
48
49         vector_results = self.store.vector_search(query_vector, limit=limit * 2)
50         keyword_results = self.store.keyword_search(query, limit=limit * 2)
51         recent_results = self.store.recent(limit=3)
52
53         scores = {}
54
55         for item, semantic_score in vector_results:
56             scores[item.id] = {
57                 "item": item,
58                 "score": semantic_score * 0.55,
59             }
60
61         for item, bm25_rank in keyword_results:
62             if item.id not in scores:
63                 scores[item.id] = {
```

```
python (tiếp)

64         "item": item,
65         "score": 0.0,
66     }
67
68     scores[item.id]["score"] += 0.20
69
70     for item in recent_results:
71         if item.id not in scores:
72             scores[item.id] = {
73                 "item": item,
74                 "score": 0.0,
75             }
76
77     scores[item.id]["score"] += 0.10
78
79     for memory_id in list(scores.keys()):
80         item = scores[memory_id]["item"]
81         scores[memory_id]["score"] += item.importance * 0.15
82
83     ranked = sorted(
84         scores.values(),
85         key=lambda x: x["score"],
86         reverse=True,
87     )
88
89     return [x["item"] for x in ranked[:limit]]
90
91     def list_memories(self) -> list[MemoryItem]:
92         return self.store.all_memories()
93
94     def _extract_explicit_memory(self, user_text: str) -> MemoryItem | None:
95         lowered = user_text.lower()
96
97         triggers = [
98             "nhớ rằng",
99             "hãy nhớ",
100            "từ nay",
101            "sau này",
102            "going forward",
103            "remember that",
104        ]
105
106         if not any(trigger in lowered for trigger in triggers):
107             return None
108
109         return MemoryItem(
110             content=f"User explicit preference/instruction: {user_text}",
111             memory_type="preference",
112             importance=0.95,
113             tags=["explicit", "user-preference"],
114             metadata={
115                 "extracted_at": datetime.now(timezone.utc).isoformat()
116             },
117         )
```

Cơ chế:

- `save_turn()` luôn lưu lượt hội thoại.
- Nếu user dùng từ như “từ nay”, “hãy nhớ”, agent lưu thêm một memory quan trọng.
- `retrieve()` kết hợp:

- semantic search,
- keyword search,
- recent memory,
- importance.

6.7. agent/prompt_builder.py

Input:

```
python
1 user_input
2 short_history
3 retrieved_memories
```

Output:

```
python
1 messages gửi vào Qwen
```

```
python
1 from memory.schema import MemoryItem
2
3
4 class PromptBuilder:
5     def build(
6         self,
7         user_input: str,
8         short_history: list[dict],
9         memories: list[MemoryItem],
10    ) -> list[dict]:
11
12        memory_block = self._format_memories(memories)
13
14        system_prompt = f"""
15        Bạn là một AI agent có memory.
16
17        Nguyên tắc:
18        - Trả lời bằng tiếng Việt.
19        - Giải thích rõ cơ chế, input, output nếu người dùng hỏi về code.
20        - Chỉ dùng memory khi nó liên quan trực tiếp.
21        - Nếu memory không chắc chắn, hãy nói rõ là thông tin được suy ra từ lịch sử trước đó.
22
23        Relevant memory:
24        {memory_block}
25        """.strip()
26
27        messages = [
28            {
29                "role": "system",
30                "content": system_prompt,
31            }
32        ]
33
34        messages.extend(short_history)
35
36        messages.append({
37            "role": "user",
38            "content": user_input,
39        })
40
41        return messages
42
43    def _format_memories(self, memories: list[MemoryItem]) -> str:
44        if not memories:
45            return "Không có memory liên quan."
46
47        lines = []
48
49        for idx, memory in enumerate(memories, start=1):
50            lines.append(
51                f"{idx}. [{memory.memory_type}] "
52                f"importance={memory.importance:.2f}\n"
53                f"{memory.content}"
54            )
55
56        return "\n\n".join(lines)
```

Cơ chế:

- Memory không tự động có tác dụng.
- Memory phải được đưa vào prompt.

- Prompt builder quyết định đưa memory vào phần nào.

6.8. agent/memory_agent.py

Input:

 python

```
1 user_input: str
```

Output:

 python

```
1 assistant_answer: str
```

```
python
1 from config import settings
2 from agent.prompt_builder import PromptBuilder
3 from llm.qwen_local import QwenLocalLLM
4 from memory.memory_manager import MemoryManager
5
6
7 class MemoryAgent:
8     def __init__(self):
9         self.llm = QwenLocalLLM()
10        self.memory = MemoryManager()
11        self.prompt_builder = PromptBuilder()
12        self.short_history: list[dict] = []
13
14    def run(self, user_input: str) -> str:
15        retrieved_memories = self.memory.retrieve(user_input)
16
17        messages = self.prompt_builder.build(
18            user_input=user_input,
19            short_history=self.short_history,
20            memories=retrieved_memories,
21        )
22
23        assistant_answer = self.llm.generate(messages)
24
25        self._update_short_history(user_input, assistant_answer)
26
27        self.memory.save_turn(user_input, assistant_answer)
28
29        return assistant_answer
30
31    def _update_short_history(self, user_input: str, assistant_answer: str):
32        self.short_history.append({
33            "role": "user",
34            "content": user_input,
35        })
36
37        self.short_history.append({
38            "role": "assistant",
39            "content": assistant_answer,
40        })
41
42        max_messages = settings.max_history_turns * 2
43
44        if len(self.short_history) > max_messages:
45            self.short_history = self.short_history[-max_messages:]
```

Cơ chế:

1. Nhận user input.
2. Tìm memory liên quan.
3. Build prompt.
4. Gọi Qwen.
5. Cập nhật short-term memory.
6. Lưu long-term memory.

6.9. main.py

Input: câu hỏi từ terminal.

Output: câu trả lời của agent.

```
python
1 from rich.console import Console
2 from rich.panel import Panel
3
4 from agent.memory_agent import MemoryAgent
5
6
7 console = Console()
8
9
10 def main():
11     agent = MemoryAgent()
12
13     console.print(Panel.fit(
14         "Memory Agent Lab\nGõ /exit để thoát, /memories để xem memory.",
15         title="Ready",
16     ))
17
18     while True:
19         user_input = console.input("\n[bold cyan]Bạn:[/bold cyan] ").strip()
20
21         if not user_input:
22             continue
23
24         if user_input == "/exit":
25             break
26
27         if user_input == "/memories":
28             memories = agent.memory.list_memories()
29
30             for memory in memories[:20]:
31                 console.print(Panel(
32                     memory.to_display_text(),
33                     title=memory.id[:8],
34                 ))
35
36             continue
37
38         answer = agent.run(user_input)
39
40         console.print(Panel(
41             answer,
42             title="Agent",
43             border_style="green",
44         ))
45
46
47 if __name__ == "__main__":
48     main()
```

Chạy:

```
bash
1 python main.py
```

6.10. scripts/visualize_memory.py

Input: database memory.db .

Output: bảng memory trong terminal.

```
python
1 from collections import Counter
2
3 from rich.console import Console
4 from rich.table import Table
5
6 from config import settings
7 from memory.sqlite_memory import SQLiteMemoryStore
8
9
10 console = Console()
11
12
13 def main():
14     store = SQLiteMemoryStore(settings.db_path)
15     memories = store.all_memories()
16
17     counter = Counter(memory.memory_type for memory in memories)
18
19     summary_table = Table(title="Memory Summary")
20     summary_table.add_column("Memory Type")
21     summary_table.add_column("Count")
22
23     for memory_type, count in counter.items():
24         summary_table.add_row(memory_type, str(count))
25
26     console.print(summary_table)
27
28     detail_table = Table(title="Recent Memories")
29     detail_table.add_column("Type")
30     detail_table.add_column("Importance")
31     detail_table.add_column("Content", overflow="fold")
32
33     for memory in memories[:10]:
34         detail_table.add_row(
35             memory.memory_type,
36             f"{memory.importance:.2f}",
37             memory.content[:200],
38         )
39
40     console.print(detail_table)
41
42
43 if __name__ == "__main__":
44     main()
```

Chạy:

```
bash
1 python scripts/visualize_memory.py
```


7 Test thực hành

Sau khi chạy python main.py , nhập lần lượt:

```
1 Từ nay khi giải thích code, hãy luôn nói rõ input và output từng file.
```

Sau đó:

```
1 Tôi đang học Agentic AI và muốn tự code từ đầu.
```

Sau đó thoát:

```
1 /exit
```

Mở lại:

```
1 python main.py
```

Hỏi:

```
1 Bạn còn nhớ tôi muốn bạn giải thích code theo kiểu nào không?
```

Kỳ vọng:

```
1 Agent trả lời rằng bạn muốn giải thích rõ input/output từng file.
```

8 Bài tập nâng cấp theo từng level

Level 1 - Memory cơ bản

Yêu cầu:

- Thêm lệnh /recent .
- Hiển thị 5 memory mới nhất.
- Không dùng embedding.

Bạn sẽ hiểu:

```

1 Memory = lưu + đọc lại.

```

Level 2 - Semantic search

Yêu cầu:

- Thêm lệnh /search nội dung .
- Cho phép tìm memory liên quan bằng vector.

Ví dụ:

```

1 /search cách tôi thích học code

```

Bạn sẽ hiểu:

```

1 Memory = lưu + biểu diễn nghĩa + truy xuất theo nghĩa.

```

Level 3 - Hybrid search

Yêu cầu:

- In ra debug score:

```

1 semantic_score
2 keyword_bonus
3 importance_score
4 recency_score
5 final_score

```

Bạn sẽ hiểu vì sao một memory được chọn.

Level 4 - LLM-based memory writer

Thêm file:

```

1 memory/llm_memory_writer.py

```

Cơ chế:

```

1 Input:
2 - user_input
3 - assistant_answer
4
5 Output JSON:
6 {
7   "should_write": true,
8   "memory_type": "preference",
9   "content": "...",
10  "importance": 0.9,
11  "tags": ["coding_style"]
12 }
```

Prompt mẫu:

```

1 MEMORY_WRITER_PROMPT = """
2 Bạn là memory writer cho AI agent.
3
4 Nhiệm vụ:
5 - Chỉ lưu thông tin có giá trị dài hạn.
6 - Không lưu câu hỏi ngắn hạn.
7 - Không lưu thông tin nhạy cảm nếu user không yêu cầu rõ.
8 - Trả về JSON hợp lệ.
9
10 Schema:
11 {
12   "should_write": true/false,
13   "memory_type": "fact|preference|project|reflection",
14   "content": "...",
15   "importance": 0.0 đến 1.0,
16   "tags": ["..."]
17 }
18 """
```

Bạn sẽ hiểu:

```

1 Memory không chỉ là database.
2 Memory là một policy.
```

Level 5 - Reflection memory

Yêu cầu:

Cứ mỗi 20 lượt hội thoại, agent tóm tắt lại:

```
text
1 - User đang học gì?
2 - User thích phong cách giải thích nào?
3 - User đang làm project gì?
4 - Những mục tiêu dài hạn là gì?
```

Lưu thành:

```
python
1 memory_type="reflection"
2 importance=0.85
```

Bạn sẽ hiểu:

```
text
1 Memory tốt không phải lưu nhiều, mà là cô đọng đúng.
```

Level 6 - Memory evaluation

Tạo file:

```
text
1 eval_memory.py
```

Dataset test:

```
python
1 TEST_CASES = [
2     {
3         "setup": [
4             "Từ nay hãy giải thích code theo kiểu input-output từng file.",
5             "Tôi đang học Agentic AI bằng local LLM."
6         ],
7         "question": "Hãy hướng dẫn tôi học memory agent.",
8         "expected_keywords": ["input", "output", "local LLM", "Agentic AI"]
9     }
10 ]
```

Đánh giá:

```

text
1 memory_hit_rate
2 expected_keyword_recall
3 answer_groundedness
    
```

Bạn sẽ hiểu:

```

text
1 Memory phải đo được, không chỉ cảm giác là nhớ.
    
```

9 Những lỗi tư duy thường gặp khi học memory

Lỗi 1: Lưu mọi thứ

Sai vì memory sẽ nhiễu. Retrieval trả về nhiều thông tin rác.

Cách sửa:

```

text
1 Chỉ lưu:
2 - sở thích dài hạn,
3 - mục tiêu học tập,
4 - thông tin project,
5 - quyết định quan trọng,
6 - lỗi đã xử lý,
7 - cấu hình môi trường.
    
```

Lỗi 2: Chỉ dùng vector search

Vector search đôi khi kém với:

```

text
1 tên file
2 mã lỗi
3 tên model
4 version package
5 đường dẫn
    
```

Cách sửa:

```

text
1 Dùng hybrid search: vector + keyword + recency + importance.
    
```

Lỗi 3: Inject quá nhiều memory vào prompt

Sai vì LLM bị loãng thông tin.

Cách sửa:

```
text
1 Chỉ lấy top 3-5 memory.
2 Mỗi memory nên ngắn.
3 Reflection memory nên được ưu tiên hơn raw history.
```

Lỗi 4: Không phân loại memory

Nên có ít nhất:

```
text
1 turn      : lượt hội thoại thô
2 fact      : thông tin sự thật về project/học tập
3 preference : sở thích, quy tắc user muốn
4 reflection : tóm tắt cấp cao
```

10

Lộ trình học đề xuất trong 10 buổi

Buổi	Chủ đề	Sản phẩm
1	Context vs memory	Agent nhớ 4 lượt gần nhất
2	SQLite memory	Lưu hội thoại qua nhiều lần chạy
3	Memory schema	Phân loại turn/fact/preference
4	Embedding	Semantic search memory
5	Hybrid retrieval	Vector + keyword + recency
6	Prompt injection memory	Đưa memory vào prompt đúng cách
7	Memory writer	Tự quyết định cái gì nên lưu
8	Reflection	Tóm tắt nhiều memory thành insight
9	Evaluation	Test memory có giúp trả lời tốt hơn không
10	Production hóa	Logging, config, CLI, debug, dashboard

11 Thứ tự nên code

Nên code theo thứ tự này:

```
1 1. config.py
2 2. llm/qwen_local.py
3 3. memory/schema.py
4 4. memory/sqlite_memory.py
5 5. memory/embedder.py
6 6. memory/memory_manager.py
7 7. agent/prompt_builder.py
8 8. agent/memory_agent.py
9 9. main.py
10 10. scripts/visualize_memory.py
```

Lý do:

- Có config trước để mọi file dùng chung.
- Có LLM trước để chắc model chạy được.
- Có schema trước để thống nhất memory format.
- Có SQLite trước để lưu được.
- Có embedder sau để thêm semantic search.
- Có manager để gom logic memory.
- Có prompt builder để inject memory.
- Có agent để nối toàn bộ pipeline.
- Có main để test.
- Có visualize để debug.

12 Kết luận định hướng

Bạn nên học memory agent theo 3 lớp:

```
1 Lớp 1: Memory storage
2 Lưu ở đâu? Lưu định dạng gì?
3
4 Lớp 2: Memory retrieval
5 Khi nào lấy lại? Lấy bằng keyword, vector hay hybrid?
6
7 Lớp 3: Memory policy
8 Cái gì nên nhớ? Cái gì nên quên? Khi nào tóm tắt?
```

Bộ code trên là bản nền tảng tốt để bạn phát triển tiếp thành:



```
1 AI Academic Reading Agent
2 AI Clinic Triage Agent
3 Course Trial Booking Agent
4 Agent học tập cá nhân
5 Agent quản lý project/codebase
```

Điểm quan trọng nhất: memory không phải là nhét toàn bộ lịch sử vào prompt. Memory tốt là hệ thống biết chọn, nén, tìm lại và sử dụng đúng lúc.